

Prompt de Desenvolvimento: Sistema de Chat Seguro (SSE)

**Instruções para o Assistente (Antigravity):** Aja como um Desenvolvedor Full-Stack Sênior. Use as especificações abaixo para gerar o código do backend e do frontend passo a passo.

## 1. Visão Geral e Arquitetura

Sistema de chat em tempo real focado em privacidade e bypass de firewalls corporativos via **Server-Sent Events (SSE)**. O acesso é feito apenas por URL e senha da sala.

| Camada              | Tecnologia Definida                                                           |
|---------------------|-------------------------------------------------------------------------------|
| Backend API         | FastAPI (Python) com endpoints REST e StreamingResponse para SSE.             |
| Frontend UI         | React, Vite, Tailwind CSS.                                                    |
| Banco de Dados      | PostgreSQL hospedado no <b>Supabase</b> (para armazenamento de texto/hashes). |
| Storage de Arquivos | <b>Cloudflare R2</b> (para imagens, evitando consumo do banco).               |

## 2. Regras de Negócio e Funcionalidades Críticas

- **Acesso Zero-Login:** O usuário digita o nickname e a senha da sala para entrar. O backend devolve um JWT temporário.
- **Compressão Client-Side (Obrigatório):** Antes de enviar qualquer imagem ao backend/R2, o frontend deve usar a biblioteca browser-image-compression para reduzir o arquivo para **menos de 1 MB** silenciosamente.
- **Renderização Automática de Imagens:** O frontend deve usar uma função Regex para detectar links terminados em .png, .jpg, .gif nas mensagens e convertê-los automaticamente em tags <img>.
- **Markdown e Código:** Renderização de blocos de código com react-markdown e react-syntax-highlighter.
- **Notificações Push:** Uso da Web Notifications API quando a aba estiver inativa (document.hidden).

### 3. Modelagem de Dados (Supabase/PostgreSQL)

Tabela: salas

- id (uuid, pk)
- nome\_url (varchar, unique)
- hash\_senha (varchar)
- created\_at (timestamp)

Tabela: mensagens

- id (uuid, pk)
- sala\_id (uuid, fk -> salas.id)
- autor\_nickname (varchar)
- conteudo (text)
- created\_at (timestamp)

### 4. Passos para Geração de Código

1. **Passo 1:** Inicializar o projeto FastAPI, configurar a conexão com o PostgreSQL (Supabase) via SQLAlchemy ou asyncpg e criar o endpoint de gerar JWT/autenticar sala.
2. **Passo 2:** Implementar o endpoint SSE no FastAPI (StreamingResponse) que escuta novas mensagens no banco e envia para os clientes conectados.
3. **Passo 3:** Criar o endpoint de upload de imagens (recebe o arquivo já comprimido e envia para o Cloudflare R2 usando `boto3`).
4. **Passo 4:** Inicializar o Frontend em React + Vite. Criar os componentes de Login da Sala e a Interface de Chat com Tailwind.
5. **Passo 5:** Implementar a lógica de interceptação de upload (browser-image-compression) e a formatação automática de mensagens (Regex + Markdown).